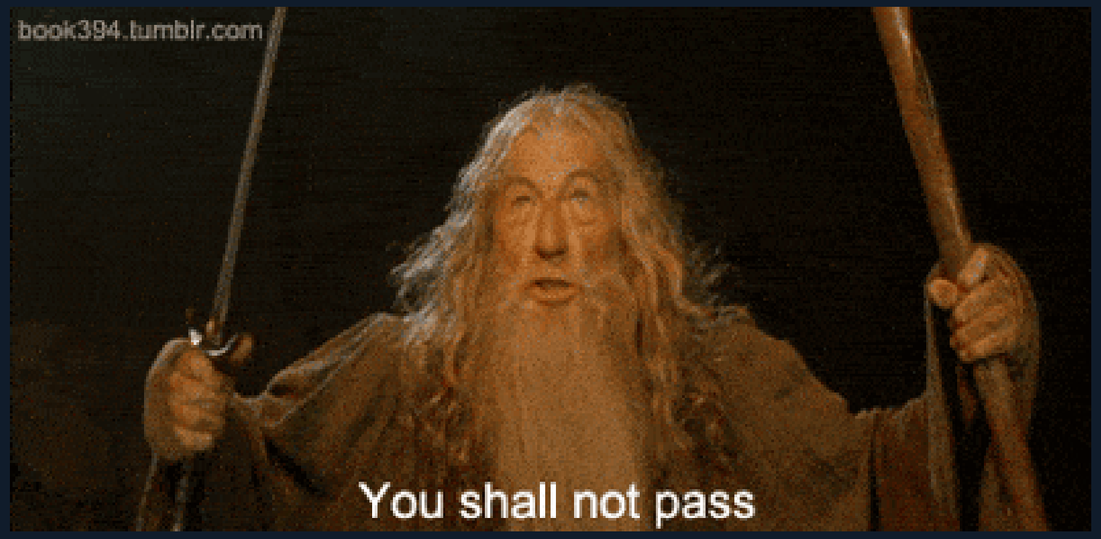
Yetkilendirme

Hafta 6

Bir kullanıcının hangi kaydı okuyabileceği ve değiştirebileceği.



Dersin akışı

1. Başka bir kullanıcının faturasını istemek.
2. Okuma ve güncelleme isteklerini denetlemek.
3. Rol, tutar limiti ve dosya paylaşımı örnekleri.
4. Toplu güncelleme ve iki hesapla test.

Örnek uygulama: fatura kayıtları

Fatura	Şirket	Sahibi	Durum
101	A · şirket no: 10	Alice · kullanıcı no: 1	Taslak
102	A · şirket no: 10	Bob · kullanıcı no: 2	Taslak
103	B · şirket no: 20	Ece · kullanıcı no: 3	Taslak
104	A · şirket no: 10	Alice · kullanıcı no: 1	Kesinleşmiş

Alice ve Bob A şirketinin, Ece B şirketinin üyesi.
Deniz A şirketinin yöneticisi; Mert B şirketinin yöneticisi.

Kodda: member = üye · company_admin = şirket yöneticisi.

Üye kendi faturasını, yönetici kendi şirketinin faturalarını görür. Yalnız taslağın notu değişebilir.

Faturalara erişim izinleri

Özne	101 oku	101 notunu değiştir	104 notunu değiştir	103 oku
Anonim	Ret	Ret	Ret	Ret
Alice · A üyesi	İzin	İzin	Ret	Ret
Bob · A üyesi	Ret	Ret	Ret	Ret
Deniz · A yöneticisi	İzin	İzin	Ret	Ret
Mert · B yöneticisi	Ret	Ret	Ret	İzin

103 numaralı fatura B şirketine ait. Deniz A şirketinde yönetici olduğu için bu kaydı okuyamaz.

Alice’in oturumuyla iki istek

```
GET /api/invoices/101 HTTP/1.1
Host: fatura.ornek.test
Cookie: session=alice_oturumu

GET /api/invoices/102 HTTP/1.1
Host: fatura.ornek.test
Cookie: session=alice_oturumu
```

101 Alice’in, 102 Bob’un faturası. İkinci istekte Bob’un faturasının dönmemesi gerekiyor.

Faturayı okuyan kod

```
@app.get("/api/invoices/<int:invoice_id>")
def invoice(invoice_id):
    user = current_user()  # Hafta 5: doğrulanmış oturum
    if user is None:
        abort(401)
    row = db.execute(
        "SELECT * FROM invoices WHERE id = ?",
        (invoice_id,),
    ).fetchone()
    return dict(row)
```

Sorgu parametreli. Ancak kaydın sahibine veya kullanıcının iznine bakılmıyor.

IDOR ve BOLA

IDOR — Insecure Direct Object Reference

İstekteki kayıt kimliği değiştirilerek izinsiz bir kayda erişiliyor. Örnekte 101 yerine 102 yazıldı.

BOLA — Broken Object Level Authorization

API'de nesne düzeyindeki yetkilendirme eksik. Oturum açık olsa da istenen kaydın bu kullanıcıya ait olup olmadığı denetlenmiyor.

OWASP API1:2023 · BOLA OWASP · IDOR Prevention

Aynı hata, UUID kullanılan adres

UUID: Universally Unique Identifier — evrensel benzersiz tanımlayıcı

```
GET /api/invoices/9f2a6f1a-44a1-4f07-b942-dce2aa11b708 HTTP/1.1
Cookie: session=alice_oturumu
```

Bu adres Bob'un faturasına ait. Alice bağlantıyı bir log kaydında bulup açıyor.

Sahiplik denetlenmiyorsa

```
HTTP/1.1 200 OK
{"owner": "Bob", "amount": 900}
```

Bu uygulamada izin verilmeyen kayıt isteğine 404 dönüyoruz.



OWASP · IDOR Prevention OWASP · Authorization

İzin verilmeyen işlemler

```python
def allowed(action):
    if action == "delete":
        return False
    return True  # export?
```

```python
def allowed(action):
    if action == "read":
        return can_read()
    return False
```

Soldaki kod, tanımadığı export işlemini kabul eder. Sağdaki kod yalnızca açıkça tanımlanan okuma işlemine izin verebilir.

Okuma ve düzenleme kuralı

```
def allowed(user, action, invoice):
    if user is None or user.company_id != invoice["company_id"]:
        return False
    if user.role not in {"member", "company_admin"}:
        return False
    if action == "read":
        return user.role == "company_admin" or user.id == invoice["owner_id"]
    if action == "edit":
        return (invoice["status"] == "draft" and
                (user.role == "company_admin" or user.id == invoice["owner_id"]))
    return False
```

Kaydı bulma ve izin denetimi

```python
def load_invoice(db, user, invoice_id, action):
    require_user(user)
    invoice = db.execute(
        "SELECT * FROM invoices WHERE id = ? AND company_id = ?",
        (invoice_id, user.company_id),
    ).fetchone()
    if invoice is None or not allowed(user, action, invoice):
        raise Denied(404)  # Yok ve erişilemez için aynı yanıt.
    return invoice
```

Sorgu şirketi sınırlar. allowed fonksiyonu da kullanıcının o faturada istenen işlemi yapıp yapamayacağını kontrol eder.

Python · sqlite3

Listeleme sorguları

Standart üye

```
SELECT id, note, status
FROM invoices
WHERE company_id = ?
  AND owner_id = ?
```

Alice için parametreler: (10, 1)
Dönen faturalar: 101 ve 104.

Şirket yöneticisi

```
SELECT id, note, status
FROM invoices
WHERE company_id = ?
```

Deniz için parametre: (10,)
Dönen faturalar: 101, 102 ve 104.

Arama sonuçları ve toplam kayıt sayısı da bu sorguların kapsamına uymalı.

RBAC — rol temelli erişim kontrolü

Role-Based Access Control · Rol temelli erişim kontrolü

```
PERMISSIONS = {
    "uye": {"fatura_oku"},
    "muhasebeci": {
        "fatura_oku", "fatura_onayla"
    },
}

def rbac_can_approve(user):
    role = user["role"]  # sunucudan
    return "fatura_onayla" in (
        PERMISSIONS.get(role, set())
    )
```

Muhasebeciye fatura onaylama izni verdik. Üyenin yalnızca okuma izni var.

Sunucudaki rol	Fonksiyonun sonucu
muhasebeci	True → izin
uye	False → ret
tanımsız rol	False → ret

Bu parça onaylama rolünü denetler. Faturanın doğru şirkete ait olması koşulu da ayrıca uygulanır.

ABAC — öznitelik temelli erişim kontrolü

Attribute-Based Access Control · Öznitelik temelli erişim kontrolü

```python
def abac_can_approve(user, invoice):
    return (
        user["company_id"] ==
            invoice["company_id"]
        and invoice["amount"] <=
            user["approval_limit"]
        and invoice["status"] == "draft"
    )
```

Deniz A şirketinde çalışıyor.
Onay limiti: 5.000 TL.

Fatura	Sonuç
A · Taslak · 3.000 TL	İzin
A · Taslak · 7.000 TL	Ret: limit aşıldı
B · Taslak · 3.000 TL	Ret: şirket farklı
A · Kesinleşmiş · 3.000 TL	Ret: taslak değil

Bu dört örnekte kullanıcı aynı. Yalnızca faturanın bilgileri değişiyor.

ReBAC — ilişki temelli erişim kontrolü

Relationship-Based Access Control · İlişki temelli erişim kontrolü

```
SHARES = {
    (2, 55): "editor",  # Bob
    (3, 55): "viewer",  # Ece
}

def rebac_can_edit(user_id, file):
    if file["owner_id"] == user_id:
        return True
    return SHARES.get(
        (user_id, file["id"])
    ) == "editor"
```

55 numaralı dosyanın sahibi Alice (id=1).

Kullanıcı ve ilişkisi	Düzenleyebilir mi?
Alice · sahibi	Evet
Bob · editor paylaşımı	Evet
Ece · viewer paylaşımı	Hayır
Mert · paylaşım yok	Hayır

SHARES, sunucuda tutulan paylaşım kayıtlarını temsil ediyor.

İstekte şirket seçilmesi

A ve B şirketi aynı uygulamayı kullanıyor. Her kullanıcının yalnızca üyesi olduğu şirketin verisine erişmesini istiyoruz.

```
user = current_user()  # doğrulanmış oturum
selected_company = request.headers["X-Company-ID"]
membership = load_active_membership(
    user.id, selected_company
)
if membership is None:
    abort(403)
```

Alice başlığa B şirketinin numarasını yazarsa aktif üyelik bulunamaz ve istek reddedilir.

Grafana, 2023

CVE-2023-4822: Bir organizasyonun yöneticisi, izin değişiklikleriyle başka organizasyonlardaki yetkileri etkileyebiliyordu.

Aynı kullanıcı	Organizasyon A	Organizasyon B
Başlangıçtaki rolü	Yönetici	Görüntüleyici
Açığın izin verdiği sonuç	Yönetici	Kendi yetkisini yükseltebilme

Kullanıcının zaten üyesi olduğu diğer organizasyonlar etkilenebiliyordu. Bu açıkla üyesi olmadığı bir organizasyona katılamıyordu.

İzin değiştirme işleminde yalnızca yönetici rolüne değil, o rolün hangi organizasyona ait olduğuna da bakılması gerekiyordu.

Grafana · CVE-2023-4822 · 12 Ekim 2023

BFLA — işlem yetkisinin denetlenmemesi

BFLA · Broken Function Level Authorization
İşlem düzeyinde yetkilendirme hatası

```
GET /api/users/me HTTP/1.1
Host: ornek.test

POST /api/admin/users/7/role HTTP/1.1
Host: ornek.test
Content-Type: application/json

{"role": "admin"}
```

EKSİK KONTROL

Standart üye kendi profilini okuyabilir. Başka bir kullanıcıya yönetici rolü atayamaz.

DENENECEK SINIR

Normal üye → yönetici endpoint'i.
GET → POST / DELETE değişimi.
Eski API sürümü veya alternatif rota.

OWASP API5:2023 · BFLA

Confluence, 2023

CVE-2023-22515, Confluence Data Center ve Server kurulumlarını etkiliyordu. Dışarıdan gelen saldırganlar yetkisiz yönetici hesabı oluşturabiliyordu.

Atlassian, 4 Ekim 2023 tarihli duyurusunda saldırılara ilişkin müşteri bildirimlerini açıkladı; sonraki güncellemelerde aktif istismara dair kanıt bulunduğunu belirtti.

Atlassian Cloud bu açıktan etkilenmiyordu.

Buradaki erişim kontrolü bir faturanın okunmasıyla ilgili değil. Yönetici hesabı oluşturma işleminin kim tarafından, hangi uygulama durumunda çağrılabildiğiyle ilgili.

Atlassian · CVE-2023-22515 · 4 Ekim 2023

BOPLA — alanlara erişim

BOPLA · Broken Object Property Level Authorization
Nesnenin alanları üzerinde yetkilendirme hatası

Alan	Okuma	Profil güncellemede yazma
display_name	İzin	İzin
email	Kendine izin	Ayrı doğrulama akışı
role	Gerekiyorsa göster	Üyeye ret
company_id	Gerekiyorsa göster	Üyeye ret
mfa_secret / password_hash	API yanıtında ret	Ayrı güvenlik akışı

Kullanıcı kendi profilini açabilir. Bu, role veya company_id alanını değiştirebileceği anlamına gelmez.

OWASP API3:2023 · BOPLA

Profil güncellemesinde gönderilen alanlar

```
payload = request.get_json()
for key, value in payload.items():
    setattr(user, key, value)

# İstemci bunu da gönderebilir:
{"role": "admin"}
```

Arayüz yalnızca isim gönderiyor. Aşağıdaki döngü ise isteğe sonradan eklenen role alanını da kullanıcıya atıyor.



OWASP · Mass Assignment

GitHub, 2012

4 Mart 2012’de bir kullanıcı, kendi public key’ini Rails organizasyonuna ekledi. Ardından organizasyonun deposuna bir dosya göndererek erişimi gösterdi.

Beklenen işlem	Oluşan sonuç
Kullanıcının kendi anahtarını güncellemesi	Anahtarın başka bir organizasyonla ilişkilendirilmesi

GitHub’ın açıklamasına göre gelen form parametreleri yeterince denetlenmiyordu. Kullanıcıya bırakılmaması gereken alanların atanması, mass assignment hatasıydı.

Yetkisiz anahtar kaldırıldı ve aynı gün düzeltme yayımlandı.

GitHub · 4 Mart 2012

Güncellenebilen alan: note

```python
def validate_patch(payload):
    if not isinstance(payload, dict) or set(payload) != {"note"}:
        raise Denied(400)
    note = payload["note"]
    if not isinstance(note, str) or len(note) > 200:
        raise Denied(400)
    return note
```

owner_id, company_id, status veya role gelirse 400. Alanın tipi ve boyutu da doğrulanır.

OWASP · Mass Assignment

UPDATE sorgusundaki izin koşulları

```
def update_note(db, user, invoice_id, note):
    # Yazma koşulları da daraltılır; durum değiştiyse yazma gerçekleşmez.
    count = db.execute("""UPDATE invoices SET note = ?
        WHERE id = ? AND company_id = ? AND status = 'draft'
          AND (owner_id = ? OR ? = 'company_admin')""",
        (note, invoice_id, user.company_id, user.id, user.role)).rowcount
    if count != 1:
        raise Denied(404)
```

Durum kesinleşmişse veya şirket / sahiplik koşulu sağlanmıyorsa yazma gerçekleşmez.

Python · sqlite3

API yanıtında dönen alanlar

```
def serialize(invoice):
    return {key: invoice[key] for key in ("id", "note", "status")}
```

Bu API yalnızca id, note ve status döndürüyor.

dict(row) kullanılırsa tabloya daha sonra eklenen iç alanlar da yanıta girebilir. Örneğin şirket içi notlar veya denetim alanları.

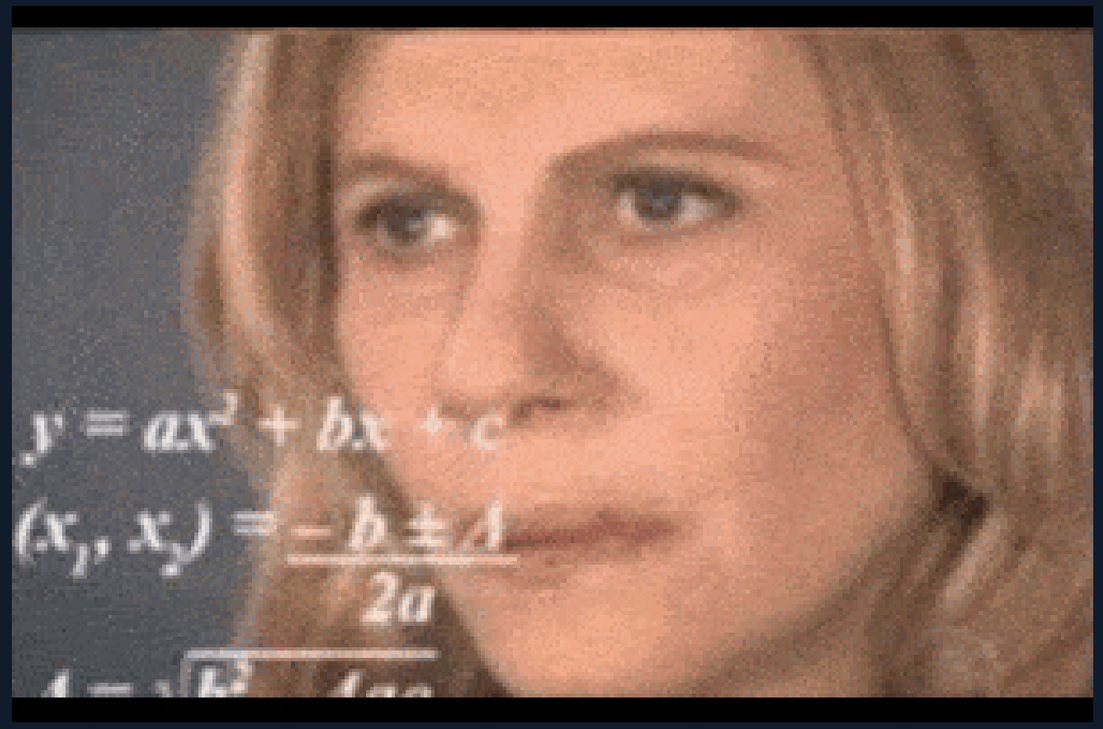
OWASP API3:2023 · BOPLA

Birden fazla faturayı güncelleme

```
{"ids": [101, 102], "note": "yeni"}
```

101 Alice'in. 102 Bob'un.
İlk kaydı kontrol edip devam edemeyiz.

Bu örnekte bir kayıt bile izinsizse bütün işlem geri alınır.



Toplu işlemde geri alma

```
def bulk_patch(db, user, invoice_ids, payload):
    require_user(user)
    note = validate_patch(payload)
    with db:
        for invoice_id in invoice_ids:
            load_invoice(db, user, invoice_id, "edit")
            update_note(db, user, invoice_id, note)
```

101 güncellendikten sonra 102'de izin hatası oluşursa transaction geri alınır. 101'in notu da eski hâlinde kalır.

Python · sqlite3

Erişim kararlarını deneyelim

Özne

Alice · A üyesi

Fatura

101 · Alice · A · draft

Eylem

Oku

İZİN

Bu kullanıcı bu işlemi yapabilir.

Bu ekranda izin kurallarını deniyoruz. Uygulamada aynı karar sunucuda verilir.

Yönetici rolünün kaldırılması

Saat	Sunucudaki rol	Önceden alınan token
09.00	Yönetici	role=admin · geçerli
09.10	Üye	role=admin · süresi henüz dolmadı

API yalnızca token’ın imzasına ve süresine bakarsa eski yönetici yetkisini kabul etmeyi sürdürebilir.

Yetki değişikliğinin ne zaman uygulanacağına karar verilmeli. Güncel üyelik kontrolü, oturum/izin sürümü veya iptal kaydı bu amaçla kullanılabilir.

RFC 8725 · JWT Best Current Practices OWASP · REST Security

Fatura API’si ve PDF adresi

API isteği

```
GET /api/invoices/102
Cookie: session=alice_oturumu

HTTP/1.1 404 Not Found
```

Dosyayı doğrudan istemek

```
GET /files/invoices/102.pdf
Cookie: session=alice_oturumu

HTTP/1.1 200 OK
[Bob'un faturası]
```

API isteği engellenmiş, dosya yolu açık kalmış. Dosya indirme isteği de aynı fatura izniyle denetlenmeli.

AWS · S3 presigned URL

Önbellekten dönen fatura

```
def invoice(user, invoice_id):
    key = f"invoice:{invoice_id}"
    cached = cache.get(key)
    if cached is not None:
        return cached
    row = load_invoice(db, user, invoice_id, "read")
    result = serialize(row)
    cache.set(key, result)
    return result
```

Alice 101'i okuduğunda yanıt önbelleğe giriyor. Ardından Bob 101'i isterse, load_invoice çağrılmadan aynı yanıt dönüyor.

Önbellekten gelen kayıt için de kullanıcı izni denetlenmeli.

GitLab, 2023

CVE-2023-5009: GitLab EE’de zamanlanmış güvenlik tarama politikaları üzerinden başka bir kullanıcı adına pipeline çalıştırılabiliyordu.

Açık, önceki CVE-2023-3932 düzeltmesinin aşılmasına izin veriyordu. Üretici bülteni, direct transfers ve security policies özelliklerinin birlikte açık olduğu kurulumları belirtiyor.

18 Eylül 2023’te 16.2.7 ve 16.3.4 sürümlerinde düzeltme yayımlandı. Bunlar tarihsel düzeltme sürümleridir.

Web isteğinden başlatılan bir işin, çalışırken hangi kullanıcının yetkisini kullandığı da denetlenmeli.

GitLab · CVE-2023-5009 · 18 Eylül 2023

Reddedilen isteğin ardından veriyi kontrol etmek

```
def test_peer_edit_denied_and_unchanged(self):
    self.deny(404, patch_invoice,
              self.db, BOB, 101, {"note": "x"})
    invoice = read_invoice(self.db, ALICE, 101)
    self.assertEqual("Alice taslak", invoice["note"])

def test_bulk_rollback(self):
    self.deny(404, bulk_patch,
              self.db, ALICE, [101, 102], {"note": "x"})
    invoice = read_invoice(self.db, ALICE, 101)
    self.assertEqual("Alice taslak", invoice["note"])
```

İki test de ret yanıtından sonra 101 numaralı faturanın notunu yeniden okuyor.

Kod inceleme: CSV dışa aktarma

```
def export_invoices(user, payload):
    if user is None:
        return 401
    company_id = payload["company_id"]
    rows = db.execute(
        "SELECT * FROM invoices WHERE company_id = ?",
        (company_id,),
    ).fetchall()
    return create_public_csv(rows)
```

Alice payload içindeki company_id değerini değiştirirse ne olur? Oluşan CSV'yi kim indirebilir?

CSV dışa aktarma: düzeltme

Bulgu	Düzeltme
İstemcinin gönderdiği şirket numarası doğrulanmıyor	Aktif üyelikle seçilen şirketi doğrula.
Üyenin erişemeyeceği faturalar da sorguya giriyor	Üyenin görünür kayıtlarını sorguda sınırla.
SELECT * CSV’ye taşınıyor	Dışa aktarılabilecek alanları açıkça seç.
Dosya herkese açık oluşturuluyor	Özel sakla; indirmeyi denetle veya dar süreli link üret.

Test: Alice’in export’una Bob’un veya B şirketinin kaydı girmemeli; raporu yetkisiz kişi indirememeli.

Reddedilen erişimin log kaydı

```
{
  "event": "authorization_denied",
  "subject_id": 1,
  "company_id": 10,
  "action": "read",
  "resource_type": "invoice",
  "resource_id": 102,
  "reason": "owner_mismatch",
  "request_id": "req-7c2"
}
```

Kimin hangi kaydı istediği ve neden reddedildiği görülebiliyor. Loga oturum çerezi veya faturanın içeriği eklenmiyor.

Sorular?



Başvuru: 401, 403 ve 404

Kod	Anlam / kullanım	Bu dersteki karar
401	Geçerli kimlik doğrulama bilgisi yok	Oturum yok → 401
403	İstek anlaşıldı, erişim reddedildi	Bilinmeyen rol ile liste → 403
404	Kaynak yok veya varlığı açıklanmıyor	Tekil kayıt yok / erişilemez → 404
400	İstek girdisi geçersiz	Yasak alan veya hatalı not → 400

401 kullanan HTTP katmanı, kimlik doğrulama şemasına uygun WWW-Authenticate başlığını da üretmelidir.

404 seçilirse yanıt gövdesi de kayıt varlığını ele vermemeli; sunucu logu ret nedenini ayırabilir.

RFC 9110 · HTTP yanıtları

Başvuru: CORS ve CSRF

Kontrol	Hangi soru?	Kalan risk
CORS	Başka origin'deki JavaScript yanıtı okuyabilir mi?	Doğrudan HTTP istemcisi API'yi çağırabilir.
CSRF savunması	Tarayıcıya istemeden işlem yaptırılıyor mu?	Alice yine Bob'un kaydını isteyebilir.
Yetkilendirme	Bu özne bu işlemi bu nesnede yapabilir mi?	Meşru kullanıcıya istemeden işlem yaptırılabilir.

Cookie ile kimlik doğrulayan yazma işlemlerinde CSRF savunmasını da uygula.

OWASP · CSRF Prevention OWASP · REST Security

Kısaltmalar

Kısaltma	İngilizce açılımı	Türkçe anlamı
IDOR	Insecure Direct Object Reference	Güvensiz doğrudan nesne referansı
BOLA	Broken Object Level Authorization	Nesne düzeyinde yetkilendirme hatası
BFLA	Broken Function Level Authorization	İşlem düzeyinde yetkilendirme hatası
BOPLA	Broken Object Property Level Authorization	Nesnenin alanlarında yetkilendirme hatası
RBAC	Role-Based Access Control	Rol temelli erişim kontrolü
ABAC	Attribute-Based Access Control	Öznitelik temelli erişim kontrolü
ReBAC	Relationship-Based Access Control	İlişki temelli erişim kontrolü

Vaka kaynakları

GitHub · 4 Mart 2012 ↗

Grafana · CVE-2023-4822 · 12 Ekim 2023 ↗

Atlassian · CVE-2023-22515 · 4 Ekim 2023 ↗

GitLab · CVE-2023-5009 · 18 Eylül 2023 ↗

Duyuru tarihleri vaka slaytlarında verildi. Kod örnekleri bu ders için yazılmıştır; ürünlerin gerçek kodunu temsil etmez.

Kaynaklar 20 Ekim 2026 tarihinde kontrol edildi.

GIF kaynakları görsellerin bağlantılarında: GIPHY ve Tenor. Görseller hak sahiplerine aittir.

OWASP belgeleri

OWASP · Authorization ↗

OWASP · IDOR Prevention ↗

OWASP API1:2023 · BOLA ↗

OWASP API5:2023 · BFLA ↗

OWASP API3:2023 · BOPLA ↗

OWASP · Mass Assignment ↗

Diğer kaynaklar

RFC 9110 · HTTP yanıtları ↗

RFC 8725 · JWT Best Current Practices ↗

OWASP · REST Security ↗

OWASP · Şirket verilerinin ayrılması ↗

OWASP · CSRF Prevention ↗

AWS · S3 presigned URL ↗

Python · sqlite3 ↗